

# PROJET Master 1 GIL

## Système de Gestion de Flotte de Véhicules

*Plan de réalisation sur 7 semaines*

**Université de Rouen**

**Enseignant encadrant : Lydia et Luc**

### Objectif du projet

Concevoir et développer un système complet de gestion de flotte automobile s'appuyant sur une architecture microservices moderne, robuste et évolutive, conforme aux meilleures pratiques du développement logiciel.

#### Compétences visées :

- Architecture microservices
- Conteneurisation et orchestration (Docker, Kubernetes)
- Sécurité applicative et gestion d'identité
- Communication asynchrone et event-driven
- Observabilité et monitoring distribué

**Année universitaire : 2025 – 2026**

# Table des matières

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Architecture &amp; Technologies</b>                  | <b>2</b>  |
| 1.1      | Vue d'ensemble de l'architecture . . . . .              | 2         |
| 1.1.1    | Diagramme d'architecture . . . . .                      | 2         |
| 1.2      | Stack technologique . . . . .                           | 2         |
| <b>2</b> | <b>Planning détaillé sur 7 semaines</b>                 | <b>3</b>  |
| <b>3</b> | <b>Critères d'évaluation</b>                            | <b>6</b>  |
| <b>4</b> | <b>Livrables attendus</b>                               | <b>7</b>  |
| <b>5</b> | <b>Bonnes pratiques à respecter</b>                     | <b>8</b>  |
| <b>6</b> | <b>Ressources recommandées</b>                          | <b>10</b> |
| 6.1      | Documentation officielle . . . . .                      | 10        |
| 6.2      | Livres et cours . . . . .                               | 10        |
| 6.3      | Outils de développement . . . . .                       | 10        |
| <b>7</b> | <b>Exemples de démarrage</b>                            | <b>10</b> |
| 7.1      | Exemple : Structure d'un microservice Node.js . . . . . | 10        |
| 7.2      | Exemple : Dockerfile multi-stage . . . . .              | 11        |
| 7.3      | Exemple : Kubernetes Deployment . . . . .               | 11        |
| <b>8</b> | <b>FAQ - Questions fréquentes</b>                       | <b>13</b> |
| <b>9</b> | <b>Conseils pour réussir</b>                            | <b>14</b> |
| 9.1      | Organisation d'équipe . . . . .                         | 14        |
| 9.2      | Gestion du code . . . . .                               | 14        |
| 9.3      | Tests et qualité . . . . .                              | 14        |
| 9.4      | Communication . . . . .                                 | 14        |
| 9.5      | Ressources d'apprentissage . . . . .                    | 14        |

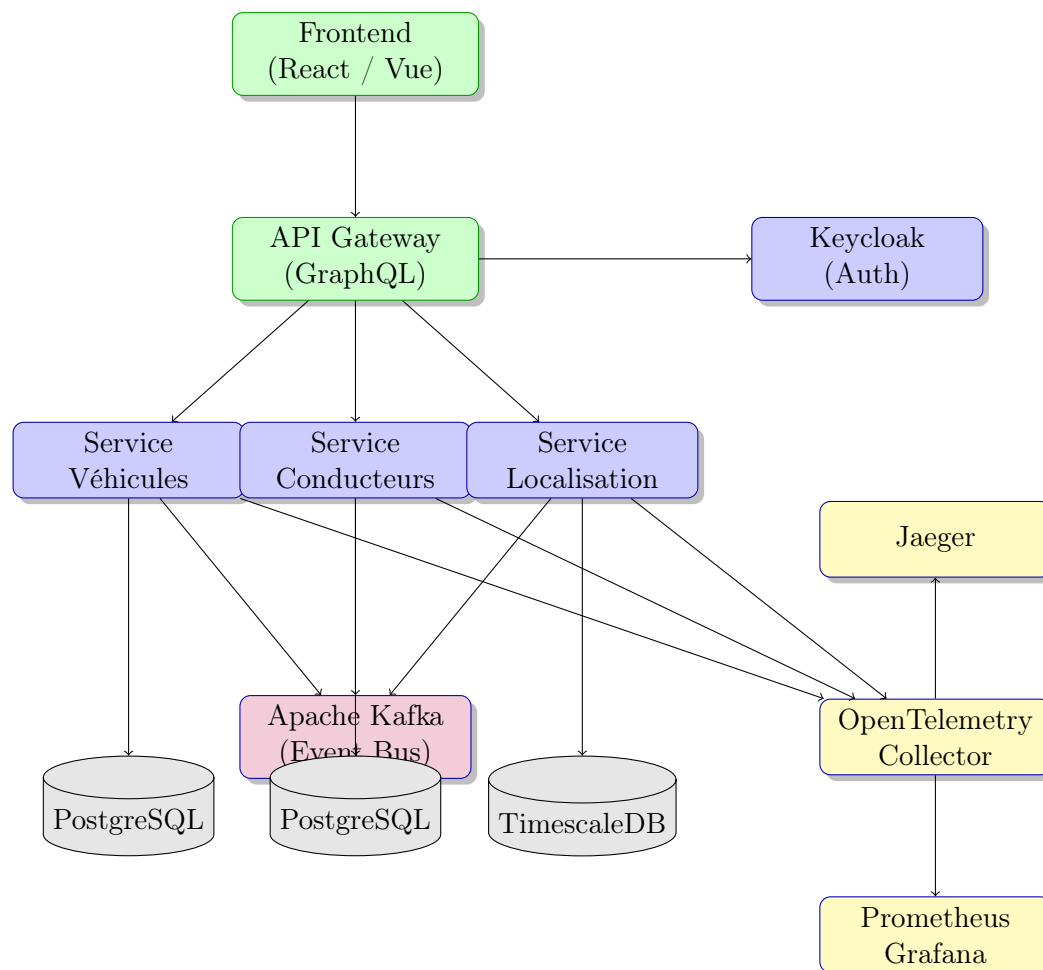
# 1 Architecture & Technologies

## 1.1 Vue d'ensemble de l'architecture

Le système repose sur une architecture microservices moderne avec 5 services principaux communiquant via une API Gateway et un bus de messages Kafka.

| Service      | Description                                                  |
|--------------|--------------------------------------------------------------|
| Véhicules    | Gestion du parc automobile (CRUD, disponibilité, historique) |
| Conducteurs  | Profils conducteurs, permis, assignations                    |
| Maintenance  | Planification et suivi des interventions                     |
| Localisation | Tracking GPS temps réel des véhicules                        |
| Événements   | Gestion des alertes et notifications                         |

### 1.1.1 Diagramme d'architecture



## 1.2 Stack technologique

**Architecture polyglotte** - choisissez les meilleurs outils pour chaque service :

| Composant       | Technologies                                                                                     |
|-----------------|--------------------------------------------------------------------------------------------------|
| Backend         | Java (Spring Boot) • Node.js (NestJS/Express) • Python (FastAPI/Django)                          |
| Base de données | PostgreSQL (principale) • Redis (cache) • TimescaleDB (localisation)                             |
| API             | REST • GraphQL (agrégation) • gRPC (temps réel)                                                  |
| Messagerie      | Apache Kafka (véhicules, déplacements, maintenance, alertes)                                     |
| Sécurité        | Keycloak (SSO, JWT, RBAC : admin, manager, technicien, utilisateur)                              |
| Frontend        | React / Vue / Angular (microfrontend)                                                            |
| Observabilité   | OpenTelemetry • Jaeger (traces) • Prometheus (métriques) • Grafana (visualisation) • Loki (logs) |
| Déploiement     | Docker • Kubernetes • Helm                                                                       |

## 2 Planning détaillé sur 7 semaines

**Note importante :** Ce planning est indicatif. Adaptez-le selon la composition de votre équipe et les difficultés rencontrées. L'important est de progresser de manière itérative et de livrer un système fonctionnel à la fin.

### Semaine 1 : Définition et conception

**Objectifs :** Poser les fondations du projet

**Durée estimée :** 15-20 heures

- Finaliser l'architecture microservices (diagrammes C4, ADR)
- Concevoir les schémas de base de données (PostgreSQL avec migrations)
- Définir les contrats d'API (OpenAPI/Swagger pour REST, schémas GraphQL)
- Configurer Keycloak (rôles, realms, clients OAuth2)
- Définir les topics Kafka et formats de messages (Avro/JSON)

**Livrables :** Documentation architecture, schémas BD, contrats API, configuration Keycloak

### Semaine 2 : Infrastructure

**Objectifs :** Mettre en place l'environnement de développement

**Durée estimée :** 15-18 heures

- Créer les Dockerfiles pour chaque service (multi-stage builds)
- Configurer Kubernetes (namespaces, deployments, services, ingress)
- Déployer PostgreSQL, Redis, Kafka avec Helm charts
- Installer la stack OpenTelemetry (Collector, Jaeger, Prometheus, Grafana, Loki)
- Configurer le CI/CD de base (GitHub Actions ou GitLab CI)

**Livrables :** Environnement Docker/K8s fonctionnel, pipeline CI/CD basique

## Semaine 3 : Microservice Véhicules

**Objectifs :** Premier service fonctionnel avec tests

**Durée estimée :** 18-22 heures

- Développer l'API REST (CRUD véhicules avec validation)
- Implémenter les événements Kafka (producer/consumer)
- Créer les résolveurs GraphQL
- Tests unitaires (>80% couverture) et d'intégration
- Instrumenter avec OpenTelemetry SDK (traces, métriques, logs structurés)

**Livrables :** Service véhicules déployé, tests passants, métriques actives

## Semaine 4 : Conducteurs et Maintenance

**Objectifs :** Enrichir l'écosystème de services

**Durée estimée :** 20-24 heures

- Service Conducteurs : CRUD, validation des permis, assignations
- Service Maintenance : planification, historique, alertes préventives
- Événements Kafka inter-services (saga pattern)
- API Gateway : agrégation GraphQL multi-services
- Intégrer Keycloak pour l'authentification

**Livrables :** 3 services communicants, API Gateway opérationnelle

## Semaine 5 : Localisation

**Objectifs :** Tracking GPS temps réel

**Durée estimée :** 18-22 heures

- Service gRPC pour le streaming de positions GPS
- Stockage time-series (TimescaleDB avec indexation spatiale)
- Intégration GraphQL pour les requêtes historiques
- Alertes géofencing via Kafka (zones autorisées/interdites)
- Simulation de données GPS pour les tests

**Livrables :** Service localisation temps réel, historique consultable

## Semaine 6 : Frontend et sécurité

**Objectifs :** Interface utilisateur sécurisée

**Durée estimée :** 22-26 heures

- Développer le microfrontend (React/Vue/Angular avec composants)
- Intégration Keycloak (SSO, login, logout)

- Gestion JWT et RBAC (routes protégées par rôle)
- Dashboards temps réel (localisation carte, état maintenance)
- Tests end-to-end (Cypress ou Playwright)

**Livrables :** Interface complète, authentification fonctionnelle

## Semaine 7 : Finalisation

**Objectifs :** Livraison et démonstration

**Durée estimée :** 15-20 heures

- Observabilité complète (Jaeger, métriques Prometheus, alertes)
- Dashboards Grafana personnalisés (business metrics)
- Tests end-to-end et tests de charge (K6 ou JMeter)
- Déploiement final avec Helm (prod-ready)
- Documentation complète (README, architecture, API docs)
- Préparation démonstration (scénarios, données de test)

**Livrables :** Système complet déployé, documentation, présentation

### 3 Critères d'évaluation

Votre projet sera évalué selon les critères suivants :

| Critère         | Description                                                   | Points     |
|-----------------|---------------------------------------------------------------|------------|
| Architecture    | Qualité de l'architecture microservices, respect des patterns | 20         |
| Code            | Qualité du code, tests, couverture, conventions               | 20         |
| Déploiement     | Docker, Kubernetes, CI/CD, automatisation                     | 15         |
| Sécurité        | Keycloak, RBAC, gestion des secrets                           | 15         |
| Fonctionnalités | Complétude des services, APIs fonctionnelles                  | 15         |
| Observabilité   | Métriques, logs, traçage, dashboards                          | 10         |
| Documentation   | Clarté, complétude, diagrammes                                | 10         |
| Démonstration   | Qualité de la présentation, maîtrise du sujet                 | 10         |
| <b>TOTAL</b>    |                                                               | <b>115</b> |

**Bonus (max +10 points) :**

- Tests de charge et optimisations (+3)
- Feature flags et canary deployment (+3)
- Internationalisation (i18n) (+2)
- Accessibilité (WCAG) du frontend (+2)

## 4 Livrables attendus

### Code source

Repository Git avec tous les microservices, scripts et configurations. Structure claire avec README par service.

### Dockerfiles

Images Docker pour chaque service, optimisées (multi-stage builds) et sécurisées (non-root user).

### Manifests Kubernetes

Deployments, Services, ConfigMaps, Secrets, Ingress. Utilisation de Helm charts recommandée.

### Configuration Keycloak

Realms, rôles, clients configurés et exportés (JSON). Script d'import automatique.

### Configuration OpenTelemetry

Fichiers de configuration du Collector, instrumentations par service, configuration des exporters.

### Dashboards Grafana

Tableaux de bord pour monitoring (CPU, mémoire, latence, événements, métriques métier). Fichiers JSON exportés.

### Documentation complète

- Architecture Decision Records (ADR)
- Guide d'installation et de déploiement
- Documentation API (OpenAPI/Swagger)
- Guide utilisateur avec captures d'écran

### Démonstration finale

Présentation live du système fonctionnel (20-30 minutes) avec scénarios de démonstration préparés.

## 5 Bonnes pratiques à respecter

### Architecture polyglotte

Choisissez le meilleur langage pour chaque service selon vos compétences et les besoins spécifiques :

- **Java/Spring Boot/Quarkus** : Services critiques nécessitant robustesse et performance
- **Node.js/NestJS** : Services I/O intensifs, temps réel, rapidité de développement
- **Python/FastAPI/FastAPi** : Data processing, ML, scripts d'automatisation

### Sécurité centralisée

#### Principe du Zero Trust :

- Utilisez Keycloak comme point unique d'authentification
- Ne gérez JAMAIS les mots de passe directement dans les services
- Implémentez le principe du moindre privilège avec RBAC
- Stockez les secrets dans Kubernetes Secrets ou Vault
- Chiffrez les communications inter-services (mTLS)

### Observabilité dès le départ

#### Les 3 piliers avec OpenTelemetry :

- **Traces** : Traçage distribué avec Jaeger (comprendre le flux des requêtes)
- **Métriques** : Prometheus (RED : Rate, Errors, Duration / USE : Utilization, Saturation, Errors)
- **Logs** : Structured logging (JSON) avec corrélation ID, agrégation avec Loki
- Ajoutez des métriques métier (véhicules actifs, alertes, etc.)

### Déploiement automatisé

#### Pipeline CI/CD :

- Build automatique des images Docker
- Tests unitaires et d'intégration obligatoires
- Scan de sécurité des images (Trivy, Snyk)
- Déploiement automatique en environnement de dev
- Utilisation de Helm pour versionner les déploiements

### Documentation continue

Documentez **au fur et à mesure**, pas à la fin !

- README.md à la racine et dans chaque service
- Architecture Decision Records (ADR) pour les choix importants
- OpenAPI/Swagger pour les APIs REST
- Commentaires dans le code (pourquoi, pas quoi)

### Approche itérative

**Fail fast, learn faster :**

- Commencez simple (MVP), enrichissez progressivement
- Livrez tôt et souvent (démonstrations hebdomadaires)
- Faites des revues de code régulières (pair programming)
- Rétrospectives d'équipe (ce qui fonctionne/ne fonctionne pas)
- Adaptez le planning si nécessaire - la flexibilité est clé !

## 6 Ressources recommandées

### 6.1 Documentation officielle

- **Microservices** : <https://microservices.io/patterns/>
- **Docker** : <https://docs.docker.com/>
- **Kubernetes** : <https://kubernetes.io/docs/>
- **Keycloak** : <https://www.keycloak.org/documentation>
- **Apache Kafka** : <https://kafka.apache.org/documentation/>
- **Prometheus** : <https://prometheus.io/docs/>
- **GraphQL** : <https://graphql.org/learn/>

### 6.2 Livres et cours

- *Building Microservices* - Sam Newman
- *Kubernetes Up & Running* - Kelsey Hightower
- *Designing Data-Intensive Applications* - Martin Kleppmann
- Cours Udemy : "Microservices with Node.js and React"
- Cours Pluralsight : "Kubernetes for Developers"

### 6.3 Outils de développement

- **IDE** : IntelliJ IDEA, VS Code, WebStorm
- **API Testing** : Postman, Insomnia
- **K8s Management** : k9s, Lens, kubectl
- **Git** : GitKraken, SourceTree, ou CLI
- **DB Management** : DBeaver, pgAdmin

## 7 Exemples de démarrage

### 7.1 Exemple : Structure d'un microservice Node.js

Listing 1 – Structure de dossiers recommandée

```
1 vehicle-service/  
2   src/  
3       controllers/      # Route handlers  
4       services/         # Business logic  
5       repositories/     # Data access  
6       models/           # Domain models  
7       config/           # Configuration  
8       middleware/       # Auth, validation, etc.  
9       app.js            # Application entry point  
10      tests/  
11          unit/  
12          integration/  
13      Dockerfile
```

```
14     .dockerignore
15     package.json
16     README.md
```

## 7.2 Exemple : Dockerfile multi-stage

Listing 2 – Dockerfile optimisé pour Node.js

```
1 # Build stage
2 FROM node:18-alpine AS builder
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci --only=production
6 COPY . .
7
8 # Production stage
9 FROM node:18-alpine
10 WORKDIR /app
11 RUN addgroup -g 1001 -S nodejs && \
12     adduser -S nodejs -u 1001
13 COPY --from=builder --chown=nodejs:nodejs /app .
14 USER nodejs
15 EXPOSE 3000
16 CMD ["node", "src/app.js"]
```

## 7.3 Exemple : Kubernetes Deployment

Listing 3 – Deployment Kubernetes basique

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: vehicle-service
5 spec:
6   replicas: 2
7   selector:
8     matchLabels:
9       app: vehicle-service
10  template:
11    metadata:
12      labels:
13        app: vehicle-service
14    spec:
15      containers:
16      - name: vehicle-service
17        image: your-registry/vehicle-service:latest
18        ports:
19        - containerPort: 3000
20        env:
21        - name: DATABASE_URL
22          valueFrom:
23            secretKeyRef:
24              name: db-secrets
25              key: url
26      resources:
27        limits:
```

```
28         cpu: "500m"
29         memory: "512Mi"
30     requests:
31         cpu: "250m"
32         memory: "256Mi"
```

## 8 FAQ - Questions fréquentes

### Q : Peut-on utiliser un seul langage pour tous les services ?

Oui, absolument ! L'architecture polyglotte est une option, pas une obligation. Si toute l'équipe maîtrise mieux un langage (par ex. Node.js), utilisez-le partout. L'important est de livrer un projet fonctionnel.

### Q : Faut-il déployer sur un vrai cluster Kubernetes ?

Non. Utilisez Minikube, Kind, ou K3s en local. Docker Desktop inclut également Kubernetes. L'important est de démontrer la conteneurisation et l'orchestration.

### Q : Comment gérer les données de test ?

Créez des scripts de seed (migrations + fixtures) pour initialiser la base avec des données cohérentes. Utilisez Faker.js ou des bibliothèques similaires pour générer des données réalistes.

### Q : Que faire si on prend du retard sur le planning ?

**Priorisez !** Concentrez-vous sur un MVP fonctionnel avec 2-3 services essentiels plutôt que 5 services incomplets. Documentez les choix de priorisation dans un ADR.

### Q : Doit-on implémenter tous les rôles RBAC ?

Minimum : 2 rôles (admin et utilisateur). Les 4 rôles complets sont un plus, mais la sécurité de base (authentification + 2 rôles) est suffisante.

### Q : Comment répartir le travail en équipe ?

Suggestion : 1 personne par service principal + rotations. Évitez les silos : faites du pair programming et des revues de code croisées pour partager les connaissances.

## 9 Conseils pour réussir

### 9.1 Organisation d'équipe

- **Travaillez en équipe** : Répartissez les microservices entre les membres
- **Daily standups** : 10-15 minutes chaque jour pour synchroniser
- **Pair programming** : Surtout pour les parties complexes (Kafka, K8s)
- **Revues de code** : Obligatoires avant tout merge en main
- **Rétrospectives** : Fin de chaque semaine, ajustez le cap

### 9.2 Gestion du code

- **Git flow** : Branches par feature, commits atomiques et descriptifs
- **Conventional Commits** : `feat:`, `fix:`, `docs:`, etc.
- **Pull Requests** : Description claire, tests passants, revue obligatoire
- **Monorepo ou multi-repo** : Les deux fonctionnent, choisissez selon vos préférences

### 9.3 Tests et qualité

- **Tests d'abord** : TDD quand possible, au minimum tests après chaque feature
- **Couverture** : Visez 70-80%, concentrez-vous sur la logique métier
- **Tests d'intégration** : Testez les interactions entre services
- **Linting** : ESLint, Prettier, etc. - configurez dès le début

### 9.4 Communication

- **Slack/Discord** : Canal dédié au projet, partage rapide
- **Documentation** : README à jour, ADR pour les décisions importantes
- **Demos** : Présentez vos avancées chaque semaine (même en interne)
- **Questions** : N'hésitez pas à demander de l'aide (profs, forums, communauté)

### 9.5 Ressources d'apprentissage

- **Documentation officielle** : Toujours la meilleure source
- **Stack Overflow** : Pour les problèmes techniques précis
- **GitHub** : Inspirez-vous de projets similaires (avec attribution !)
- **Tutoriels vidéo** : YouTube, Udemy, Pluralsight
- **Blogs techniques** : Medium, Dev.to, blogs d'entreprises tech

## Bon courage !

Ce projet est ambitieux mais vous allez acquérir des compétences essentielles en architecture distribuée. N'hésitez pas à vous entraider et à itérer sur vos solutions.

**Rappel important :** Un système simple qui fonctionne vaut mieux qu'un système complexe qui ne fonctionne pas. Commencez petit, testez souvent, et faites évoluer progressivement.

La réussite passe par la collaboration, la persévérance, et l'apprentissage continu !

## Annexe : Glossaire

|                      |                                                                                          |
|----------------------|------------------------------------------------------------------------------------------|
| <b>ADR</b>           | Architecture Decision Record - Document traçant les décisions d'architecture importantes |
| <b>API Gateway</b>   | Point d'entrée unique pour toutes les requêtes clients vers les microservices            |
| <b>CRUD</b>          | Create, Read, Update, Delete - Opérations de base sur les données                        |
| <b>gRPC</b>          | Framework RPC haute performance développé par Google                                     |
| <b>Helm</b>          | Gestionnaire de packages pour Kubernetes                                                 |
| <b>JWT</b>           | JSON Web Token - Standard pour les tokens d'authentification                             |
| <b>Kafka</b>         | Plateforme de streaming événementiel distribué                                           |
| <b>Keycloak</b>      | Solution open-source de gestion d'identité et d'accès                                    |
| <b>Microservices</b> | Architecture composée de services indépendants et faiblement couplés                     |
| <b>mTLS</b>          | Mutual TLS - Authentification mutuelle par certificats                                   |
| <b>RBAC</b>          | Role-Based Access Control - Contrôle d'accès basé sur les rôles                          |
| <b>REST</b>          | Representational State Transfer - Style d'architecture pour les APIs web                 |
| <b>Saga</b>          | Pattern de gestion des transactions distribuées                                          |
| <b>SSO</b>           | Single Sign-On - Authentification unique                                                 |
| <b>TimescaleDB</b>   | Base de données optimisée pour les séries temporelles                                    |
| <b>Zero Trust</b>    | Modèle de sécurité où rien n'est automatiquement digne de confiance                      |